

Infosys Java Multithreading Interview Prep

3 Years Experience • Questions First • Detailed Answers Separately • Interview Tomorrow

Research basis: Recent Infosys candidate reports specifically mention multithreading questions and, in one 2-year Java interview report, asking how multithreading was implemented in the candidate's project. Current 2026 interview reports also show Core Java, Spring Boot, microservices, SQL and coding being tested. Current Java concurrency guides repeatedly emphasize thread lifecycle, synchronized, volatile, deadlocks, executors, CompletableFuture and concurrent collections. These are recurring themes, not a guarantee of the exact questions tomorrow.

Sources reviewed: Infosys candidate reports on GeeksforGeeks and Glassdoor, plus current 2026 Java concurrency interview guides.

PART 1 — QUESTIONS ONLY

Answer each aloud before checking Part 2.

1. What is a process vs a thread?
2. What is multithreading and why is it useful in backend applications?
3. How can you create a thread in Java?
4. start() vs run()?
5. Explain Java thread states.
6. What is a race condition?
7. What is thread safety?
8. What does synchronized guarantee?
9. Synchronized method vs synchronized block?
10. Is synchronized reentrant?
11. What is volatile?
12. Why is volatile int count not enough for count++?
13. synchronized vs volatile?
14. What is AtomicInteger?
15. What is CAS?
16. What is deadlock?
17. How do you prevent deadlock?
18. What are starvation and livelock?
19. wait(), notify(), notifyAll()?
20. wait() vs sleep()?
21. Why use wait() inside while rather than if?
22. What is join()?
23. What is ExecutorService?
24. execute() vs submit()?
25. Why use a thread pool?
26. Runnable vs Callable?
27. Future vs CompletableFuture?
28. thenApply vs thenCompose vs thenCombine?
29. What is ReentrantLock?
30. What is ThreadLocal?
31. What is ConcurrentHashMap?
32. ConcurrentHashMap vs synchronizedMap?
33. What is BlockingQueue?
34. What is the Java Memory Model?

- 35. What is happens-before?
- 36. What is `CompletableFuture.supplyAsync` vs `runAsync`?
- 37. What is `@Async` in Spring Boot?
- 38. How does multithreading work for a Spring Boot REST API?
- 39. Two application instances update the same DB row. Is synchronized enough?
- 40. How would you process one million records safely?

PART 2 — MODEL ANSWERS

1. What is a process vs a thread?

A process is an independent program in execution with its own address space/resources. A thread is an execution unit inside a process. Threads of the same process share heap and process resources, while each thread has its own stack and program counter. Threads are generally lighter than processes.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

2. What is multithreading and why is it useful in backend applications?

Multithreading lets multiple tasks make progress concurrently. Backend servers use thread pools to handle multiple requests and background work. It can improve throughput and responsiveness, especially when work involves waiting for I/O. It does not automatically make every task faster.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

3. How can you create a thread in Java?

You can extend Thread, implement Runnable, or submit Runnable/Callable tasks to ExecutorService. For production applications, ExecutorService is usually preferred because task logic is separated from thread management and threads can be reused.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

4. start() vs run()?

start() causes a new thread of execution to be scheduled, which eventually invokes run(). Calling run() directly is an ordinary method call on the current thread and creates no new concurrency.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

5. Explain Java thread states.

NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING and TERMINATED. BLOCKED means waiting for a monitor lock; WAITING includes indefinite waits such as wait/join; TIMED_WAITING includes sleep or timed wait/join.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

6. What is a race condition?

A race condition occurs when multiple threads access shared mutable state and correctness depends on their timing. Example: two threads executing count++ can both read the same value and lose an update.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

7. What is thread safety?

Code is thread-safe when it remains correct under concurrent access according to its contract. Common approaches are immutability, synchronization, atomic classes, locks, thread confinement and concurrent collections.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

8. What does synchronized guarantee?

synchronized provides mutual exclusion for the same monitor and memory-visibility/order guarantees. A thread entering a monitor can see writes made by a previous thread that released that same monitor.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

9. Synchronized method vs synchronized block?

An instance synchronized method locks this; a static synchronized method locks the Class object. A synchronized block lets you choose the lock and usually keep the critical section smaller.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

10. Is synchronized reentrant?

Yes. A thread that already owns a monitor can acquire the same monitor again. This is called reentrancy.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

11. What is volatile?

volatile provides visibility and ordering guarantees for a variable across threads. It does not provide mutual exclusion and does not make compound operations such as count++ atomic.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

12. Why is volatile int count not enough for count++?

count++ is read, modify and write. Two threads can read the same old value and overwrite each other's increments. Use AtomicInteger/AtomicLong or a suitable lock.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

13. synchronized vs volatile?

synchronized gives mutual exclusion plus visibility/order around the lock. volatile gives visibility/order for the variable but no general atomicity or exclusive access.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

14. What is AtomicInteger?

AtomicInteger provides atomic operations such as incrementAndGet and compareAndSet. It is useful for independent counters and other single-variable state transitions without an explicit synchronized block.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

15. What is CAS?

Compare-and-set updates a value only if it still equals an expected value. It is a foundation of many atomic/non-blocking algorithms. If another thread changed the value first, CAS fails.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

16. What is deadlock?

Deadlock occurs when threads wait forever for resources held by one another. Example: Thread A holds lock1 and waits for lock2 while Thread B holds lock2 and waits for lock1.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

17. How do you prevent deadlock?

Use consistent lock ordering, minimize nested locking, keep critical sections small, and where appropriate use tryLock with timeouts. Avoid unnecessary shared locks.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

18. What are starvation and livelock?

Starvation means a thread waits excessively because other work keeps getting resources. Livelock means threads remain active and react to each other but make no useful progress.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

19. wait(), notify(), notifyAll()?

They are Object monitor methods for coordination. wait releases the object's monitor and waits. notify wakes one waiting thread; notifyAll wakes all. They must be called while holding that object's monitor.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

20. wait() vs sleep()?

wait() is for coordination and releases the associated monitor while waiting. sleep() pauses the current thread for a duration but does not release monitors it holds.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

21. Why use wait() inside while rather than if?

After waking, the condition may no longer be true, and spurious wakeups are possible. Recheck the condition in a while loop before proceeding.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

22. What is join()?

join makes the calling thread wait for another thread to finish, optionally with a timeout. It is useful when later work depends on completion of a worker.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

23. What is ExecutorService?

ExecutorService manages worker threads and executes submitted Runnable/Callable tasks. It provides pooling, Futures and lifecycle methods such as shutdown.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

24. execute() vs submit()?

execute accepts a Runnable and returns no Future. submit returns a Future and accepts Runnable or Callable. With submit, task failure is commonly observed through Future.get().

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

25. Why use a thread pool?

It reuses threads, limits concurrency and avoids creating an uncontrolled number of threads. Pool size, queue capacity and rejection behavior should match the workload and downstream capacity.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

26. Runnable vs Callable?

Runnable has no return value and run cannot declare checked exceptions. Callable returns a value and can throw checked exceptions; it is commonly submitted to an ExecutorService.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

27. Future vs CompletableFuture?

Future represents an asynchronous result but get is commonly blocking. CompletableFuture supports composition and chaining such as thenApply, thenCompose, thenCombine and exception handling.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

28. thenApply vs thenCompose vs thenCombine?

thenApply transforms a result T to U. thenCompose chains another asynchronous operation and flattens the result. thenCombine combines two independent futures after both complete.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

29. What is ReentrantLock?

It is an explicit reentrant lock with features such as tryLock, timed acquisition, interruptible locking and multiple Condition objects. It must be unlocked reliably, normally in finally.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

30. What is ThreadLocal?

ThreadLocal gives each thread its own value. It can be useful for per-thread context, but pooled threads can retain values, so cleanup with remove is important when appropriate.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

31. What is ConcurrentHashMap?

It is a thread-safe map designed for concurrent access with high concurrency. It supports atomic operations such as `putIfAbsent` and `computeIfAbsent` and does not permit null keys or values.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

32. ConcurrentHashMap vs synchronizedMap?

`synchronizedMap` generally serializes map operations through a common synchronization mechanism. `ConcurrentHashMap` is designed for concurrent access and supports atomic compound operations. Its iterators are weakly consistent.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

33. What is BlockingQueue?

`BlockingQueue` is a thread-safe queue for producer-consumer designs. `put` can wait when full and `take` can wait when empty, providing natural coordination and backpressure.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

34. What is the Java Memory Model?

The JMM defines visibility, ordering and synchronization rules for interactions between threads. It explains why unsynchronized shared state can be unsafe and how synchronized, volatile and atomic mechanisms establish guarantees.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

35. What is happens-before?

It is a JMM relationship that guarantees visibility and ordering. For example, an unlock happens-before a later lock on the same monitor, and a volatile write happens-before a subsequent read of that variable.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

36. What is CompletableFuture.supplyAsync vs runAsync?

`supplyAsync` is for an asynchronous computation that returns a value; `runAsync` is for one with no result. Both can use the common pool by default or an explicitly supplied `Executor`.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

37. What is @Async in Spring Boot?

`@Async` lets Spring execute a method asynchronously using a Spring-managed executor when async support is enabled. Configure the executor deliberately and handle exceptions/results. Self-invocation does not pass through the Spring proxy, so it does not activate `@Async`.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

38. How does multithreading work for a Spring Boot REST API?

The embedded servlet container normally has a request-thread pool. Concurrent HTTP requests can execute on different worker threads. Therefore singleton Spring beans should normally be stateless; mutable instance fields can become shared state across requests.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

39. Two application instances update the same DB row. Is synchronized enough?

No. synchronized protects threads within one JVM instance only. For database-wide concurrency, use database mechanisms such as optimistic locking with @Version, pessimistic locking where justified, or atomic SQL updates/constraints.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

40. How would you process one million records safely?

Do not create one million threads. Read records in pages/chunks or stream them, use bounded worker pools, batch DB writes, apply backpressure, monitor memory/DB connections and make processing idempotent where retries are possible. Do not blindly use parallelStream.

Likely follow-up: Explain a real project example, edge case, or trade-off. Infosys interviewers may drill from your resume/project rather than staying at definition level.

LAST-MINUTE PRIORITY LIST

- Race condition and thread safety
- synchronized: monitor, mutual exclusion, visibility
- volatile vs synchronized vs AtomicInteger
- Deadlock + prevention + thread dump
- wait/notify/notifyAll vs sleep/join
- Runnable vs Callable
- ExecutorService, thread pool, execute vs submit, shutdown
- Future vs CompletableFuture
- ReentrantLock and tryLock
- ConcurrentHashMap and BlockingQueue
- JMM + happens-before
- Spring @Async and singleton bean thread safety
- Concurrent REST requests in Spring Boot
- Database concurrency: @Version / optimistic locking
- One-million-record processing and why not blindly use parallelStream

Highest-priority questions for tonight: 6, 8, 11, 12, 16, 19, 20, 23, 24, 25, 27, 29, 31, 34, 35, 37, 38, 39, 40.